Python Data Analysis - Part 2

Table of Contents

The pandas is an open source library for data analysis in Python. Since we will also use NumPy beside pandas, first we need to import the modules.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, HTML
%matplotlib inline
import warnings
warnings.filterwarnings('ignore')
```

Pandas has three main data structures:

1. Series
2. Data Frame
3. Panel

Series

Series is one dimensional data similar to an array. Series has two main attributes: **index** and **values**.

In [2]:
```
val = [x for x in range(1, 30, 3)]  # [1, 4, 7, 10, 13, 16, 19, 22, 25, 28]
idx = [x for x in range(0, 10)]  # [0, 1, 2,  3,  4,  5,  6,  7,  8,  9]
s = pd.Series(val, index=idx)
display(s)
type(s)
```

```
0     1
1     4
2     7
3    10
4    13
5    16
6    19
7    22
8    25
9    28
dtype: int64
```

Out[2]:
```
pandas.core.series.Series
```

The index and the values are not always numbers. It can also be string.

In [3]:
```
s = pd.Series([15,-3, 9, 6], index=['z','q','c','f'])
s
```

Out[3]:
```
z    15
q    -3
c     9
f     6
dtype: int64
```

sort_values() method is useful to sort the values

In [4]:
```
s.sort_values()
```

Out[4]:
```
q    -3
f     6
c     9
z    15
dtype: int64
```

In [5]:
```
s.sort_values(ascending=False)
```

Out[5]:
```
z    15
c     9
f     6
q    -3
dtype: int64
```

To sort the index, use **sort_index()** method

In [6]:
```
s.sort_index()
```

Out[6]:
```
c     9
f     6
q    -3
z    15
dtype: int64
```

Relabel the index is simply change the index value.

In [7]:
```
val=[x for x in range(1,30,3)]    # [1, 4, 7, 10, 13, 16, 19, 22, 25, 28]
idx2=[x for x in range(1,11)]     # [1, 2, 3,  4,  5,  6,  7,  8,  9, 10]
s=pd.Series(val,index=idx2)
s
```

Out[7]:
```
1      1
2      4
3      7
4     10
5     13
6     16
7     19
8     22
9     25
10    28
dtype: int64
```

Method **reindex()** change the order of the sequence of indexes, delete some of them, or add new ones. In the case of a new label, pandas add NaN as corresponding value.

In [8]:
```
val=[x for x in range(1,30,3)]   # [1, 4, 7, 10, 13, 16, 19, 22, 25, 28]
idx=[x for x in range(0,10)]     # [0, 1, 2,  3,  4,  5,  6,  7,  8,  9]
idx2=[x for x in range(1,11)]    # [1, 2, 3,  4,  5,  6,  7,  8,  9, 10]
s=pd.Series(val,index=idx)
s1=s.reindex(idx2)
s1
```

Out[8]:
```
1      4.0
2      7.0
3     10.0
4     13.0
5     16.0
6     19.0
7     22.0
8     25.0
9     28.0
10     NaN
dtype: float64
```

Accessing the value of a series is by specifying the index

In [9]:
```
s[4]=12
s
```

Out[9]:
```
0     1
1     4
2     7
3    10
4    12
5    16
6    19
7    22
8    25
9    28
dtype: int64
```

We can filter the series. s1 comes from s with values higher than 10. s2 comes from s with values less than or equal to 16.

In [10]:
```
s1=s[s>10]
s1
```

Out[10]:
```
4    12
5    16
6    19
7    22
8    25
9    28
dtype: int64
```

In [11]:
```
s2=s[s<=16]
s2
```

Out[11]:
```
0     1
1     4
2     7
3    10
4    12
5    16
dtype: int64
```

In [12]:
```
s2.index
```

Out[12]:
```
Int64Index([0, 1, 2, 3, 4, 5], dtype='int64')
```

In [13]:
```
s2.values
```

Out[13]:
```
array([ 1,  4,  7, 10, 12, 16], dtype=int64)
```

In [14]:
```
s
```

Out[14]:
```
0     1
1     4
2     7
3    10
4    12
5    16
6    19
7    22
8    25
9    28
dtype: int64
```

In [15]:
```
s.drop(s2.index)   # remove some part of the series based on criteria on values
```

Out[15]:
```
6    19
7    22
8    25
9    28
dtype: int64
```

When we sum the two series, the summation only happens for the items with common index labels. All other index labels from the two series are copied but the values are NaN.

In [16]:
```
s3=s1+s2
s3
```

Out[16]:
```
0     NaN
1     NaN
2     NaN
3     NaN
4    24.0
5    32.0
6     NaN
7     NaN
8     NaN
9     NaN
dtype: float64
```

To remove NaN from the data, use **dropna()** method.

In [17]:
```
s3.dropna()
```

Out[17]:
```
4    24.0
5    32.0
dtype: float64
```

Another possibility to remove NaN from the data is to use **notnull()** method that produce Boolean values for each label.

In [18]:
```
s3.notnull()
```

Out[18]:
```
0    False
1    False
2    False
3    False
4     True
5     True
6    False
7    False
8    False
9    False
dtype: bool
```

Then we use these Boolean value as the mask into the Series.

In [19]:
```
s3[s3.notnull()]
```

Out[19]:
```
4    24.0
5    32.0
dtype: float64
```

To remove a certain data based on the index label, we use **drop()** method. This operation is useful to exclude certain data from the analysis.

First, we presented the Series again

In [20]:
```
val=[x for x in range(1,30,3)]   # [1, 4, 7, 10, 13, 16, 19, 22, 25, 28]
idx2=[x for x in range(1,11)]    # [1, 2, 3,  4,  5,  6,  7,  8,  9, 10]
s=pd.Series(val,index=idx2)
s
```

Out[20]:
```
1      1
2      4
3      7
4     10
5     13
6     16
7     19
8     22
9     25
10    28
dtype: int64
```

Then, we drop index 7 to 10

In [21]:
```
arr=range(7,11)
s=s.drop(arr)
s
```

Out[21]:
```
1     1
2     4
3     7
4    10
5    13
6    16
dtype: int64
```

Data Frame

Data Frame is two dimensional data similar to an matrix or a database table. Data Frame has three main attributes: **index**, **values** and **columns**.

Creating Data Frame

Series can be transformed into Data Frame

In [22]:
```
val=[x for x in range(1,30,3)]   # [1, 4, 7, 10, 13, 16, 19, 22, 25, 28]
idx=[x for x in range(0,10)]     # [0, 1, 2,  3,  4,  5,  6,  7,  8,  9]
s=pd.Series(val,index=idx)
df=s.to_frame(name='val')
display(df)
type(df)
```

	val
0	1
1	4
2	7
3	10
4	13
5	16
6	19
7	22
8	25
9	28

Out[22]:
```
pandas.core.frame.DataFrame
```

We can create a data frame from NumPy matrix

In [23]:
```
df=np.arange(1,100,4).reshape(5,5)
df
```

Out[23]:
```
array([[ 1,  5,  9, 13, 17],
       [21, 25, 29, 33, 37],
       [41, 45, 49, 53, 57],
       [61, 65, 69, 73, 77],
       [81, 85, 89, 93, 97]])
```

To know the number of rows and the number of columns, use **shape** attribute

In [24]:
```
df.shape
```

Out[24]:
```
(5, 5)
```

In [25]:
```
print('Number of rows:',df.shape[0])
print('Number of columns:',df.shape[1])
```

```
Number of rows: 5
Number of columns: 5
```

In [26]:
```
df1 = pd.DataFrame(df, columns=['Strength','Elasticity','Diameter','Elongation','Depth'
])
df1
```

Out[26]:

	Strength	Elasticity	Diameter	Elongation	Depth
0	1	5	9	13	17
1	21	25	29	33	37
2	41	45	49	53	57
3	61	65	69	73	77
4	81	85	89	93	97

The column headers names

In [27]:
```
df1.columns
```

Out[27]:
```
Index(['Strength', 'Elasticity', 'Diameter', 'Elongation', 'Depth'], dtype='object')
```

The number of columns in the data frame

In [28]:
```
len(df1.columns)
```

Out[28]:
```
5
```

In [29]:
```
df1.index
```

Out[29]:
```
RangeIndex(start=0, stop=5, step=1)
```

Number of rows

In [30]:
```
len(df1.index)
```

Out[30]:
```
5
```

Header does not change the number of rows and columns.

In [31]:
```
print('Number of rows:',df1.shape[0])
print('Number of columns:',df1.shape[1])
```

```
Number of rows: 5
Number of columns: 5
```

The values inside dataframe

In [32]:
```
df1.values
```

Out[32]:
```
array([[ 1,  5,  9, 13, 17],
       [21, 25, 29, 33, 37],
       [41, 45, 49, 53, 57],
       [61, 65, 69, 73, 77],
       [81, 85, 89, 93, 97]])
```

Reindex (change the index name) to 1 to 6 (instead of 0 to 5)

In [33]:
```
df1.index=np.arange(1,6)
df1
```

Out[33]:

	Strength	Elasticity	Diameter	Elongation	Depth
1	1	5	9	13	17
2	21	25	29	33	37
3	41	45	49	53	57
4	61	65	69	73	77
5	81	85	89	93	97

In [34]:
```
df1.index
```

Out[34]:
```
Int64Index([1, 2, 3, 4, 5], dtype='int64')
```

We can also create data frame from dictionary

In [35]:
```
dict1={'color': ['red', 'green', 'yellow'],
       'fruit': ['apple','banana','cherry'],
       'profile': ['round', 'long', 'round'],
       'taste': ['crunchy','sweet','sour']}
df2=pd.DataFrame(dict1,index=np.arange(1,4))
df2
```

Out[35]:

	color	fruit	profile	taste
1	red	apple	round	crunchy
2	green	banana	long	sweet
3	yellow	cherry	round	sour

Selection

To select one column, you can use *df['ColumnName']* or *df.ColumnName* or *df.ix[:,colNum]*. The result is a Series.

In [36]:
```
s=df2.color
type(s)
```

Out[36]:
```
pandas.core.series.Series
```

In [37]:
```
df2['fruit']
```

Out[37]:
```
1     apple
2    banana
3    cherry
Name: fruit, dtype: object
```

In [38]:
```
df2.ix[:,0]
```

Out[38]:
```
1       red
2     green
3    yellow
Name: color, dtype: object
```

To select one particular cell within the data frame, you use *df.ColumnName[index]* or *df['ColumnName'][index]*.

In [39]:
```
df2.profile[3]
```

Out[39]:
```
'round'
```

In [40]:
```
df2['taste'][2]
```

Out[40]:
```
'sweet'
```

To select one particular row wthin the data frame, use *df.**ix[index]*** or *df.**ix[rowNum,:]*** .

In [41]:
```
df2.ix[2]
```

Out[41]:
```
color       green
fruit      banana
profile      long
taste       sweet
Name: 2, dtype: object
```

In [42]:
```
s=df2.ix[2,:]
display(s)
type(s)
```

```
color       green
fruit      banana
profile      long
taste       sweet
Name: 2, dtype: object
```

Out[42]:
```
pandas.core.series.Series
```

Row selection can also use colon notation df[*start*: *end*: *step*]. The *start* is inclusive (from 0 if omitted), the *end* is exclusive (until the end of rows if omitted) and the default step value is 1 if omitted.

In [43]:
```
df2[0:2]
```

Out[43]:

	color	fruit	profile	taste
1	red	apple	round	crunchy
2	green	banana	long	sweet

To select multiple rows, provide list of index as argument to ix[index]

In [44]:
```
df2.ix[[1,3]]
```

Out[44]:

	color	fruit	profile	taste
1	red	apple	round	crunchy
3	yellow	cherry	round	sour

Assignment

Updating the value is done similar to selection.

In [45]:
```
print(df2['taste'][2])
df2['taste'][2]='sour'
print(df2['taste'][2])
```

```
sweet
sour
```

It is possible to assign the same value for one column. Suppose we create a new column and assign equal initial value.

In [46]:
```
df2['demand']=20
df2
```

Out[46]:

	color	fruit	profile	taste	demand
1	red	apple	round	crunchy	20
2	green	banana	long	sour	20
3	yellow	cherry	round	sour	20

We can also assign to all the values in one column by assigning to a list of the same length to the number of rows.

In [47]:
```
df2['demand']=[15,10,30]
df2
```

Out[47]:

	color	fruit	profile	taste	demand
1	red	apple	round	crunchy	15
2	green	banana	long	sour	10
3	yellow	cherry	round	sour	30

Filtering

In [48]:
```
print(df2.demand>10)
np.sum(df2.demand>10)  # count how manyTrue
```

```
1     True
2    False
3     True
Name: demand, dtype: bool
```

Out[48]: 2

In [49]:
```
df2[df2.demand>10]
```

Out[49]:

	color	fruit	profile	taste	demand
1	red	apple	round	crunchy	15
3	yellow	cherry	round	sour	30

In [50]:
```
df2[df2.taste=='sour']
```

Out[50]:

	color	fruit	profile	taste	demand
2	green	banana	long	sour	10
3	yellow	cherry	round	sour	30

Membership

membership function **isin([*value*, *column*])** is useful for masking

In [51]:
```
mask1=df2.isin(['round','profile'])
mask1
```

Out[51]:

	color	fruit	profile	taste	demand
1	False	False	True	False	False
2	False	False	False	False	False
3	False	False	True	False	False

In [52]:
```
df2[mask1]
```

Out[52]:

	color	fruit	profile	taste	demand
1	NaN	NaN	round	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	round	NaN	NaN

In [53]:
```
mask2=df2.isin(['sour','taste'])
mask2
```

Out[53]:

	color	fruit	profile	taste	demand
1	False	False	False	False	False
2	False	False	False	True	False
3	False	False	False	True	False

In [54]:
```
mask3=mask1.values | mask2.values
mask3
```

Out[54]:
```
array([[False, False,  True, False, False],
       [False, False, False,  True, False],
       [False, False,  True,  True, False]], dtype=bool)
```

Delete

To delete a certain cell, set it to NaN

In [55]:
```
df2['demand'][1]=np.nan
df2
```

Out[55]:

	color	fruit	profile	taste	demand
1	red	apple	round	crunchy	NaN
2	green	banana	long	sour	10.0
3	yellow	cherry	round	sour	30.0

To delete a column use **del** *df['ColumnName']*

In [56]:
```
del df2['demand']
df2
```

Out[56]:

	color	fruit	profile	taste
1	red	apple	round	crunchy
2	green	banana	long	sour
3	yellow	cherry	round	sour

To delete a row, use **drop(*index*)** function

In [57]:
```
df2.drop(3)
```

Out[57]:

	color	fruit	profile	taste
1	red	apple	round	crunchy
2	green	banana	long	sour

In [58]:
```
df2.drop([1,2])
```

Out[58]:

	color	fruit	profile	taste
3	yellow	cherry	round	sour

del and **drop()** function does not actually delete the data permanently. It is only to remove column or row for the analysis.

In [59]:
```
df2
```

Out[59]:

	color	fruit	profile	taste
1	red	apple	round	crunchy
2	green	banana	long	sour
3	yellow	cherry	round	sour

Merge

Use *on=['key1', 'key2']* if the keys we want to join are the same name. If they are non the same name, use *left_on=['key1', 'key2']* and *right_on=['Key1', 'Key2']*.

how

Merge method	SQL Join Name	Description
left	LEFT OUTER JOIN	Use keys from left frame only
right	RIGHT OUTER JOIN	Use keys from right frame only
outer	FULL OUTER JOIN	Use union of keys from both frames
inner	INNER JOIN	Use intersection of keys from both frames

Merge based on one key

Notice that the right table has repeated key (not unique)

In [60]:
```
left = pd.DataFrame({'key': ['K0', 'K1', 'K2', 'K3', 'K4', 'K5'],
                     'A': ['A0', 'A1', 'A2', 'A3', 'A4', 'A5'],
                     'B': ['B0', 'B1', 'B2', 'B3', 'B4', 'B5']})
right = pd.DataFrame({'key': ['K0', 'K4', 'K3', 'K3'],
                      'C': ['C0', 'C1', 'C2', 'C3'],
                      'D': ['D0', 'D1', 'D2', 'D3']})
```

In [61]:
```
result = pd.merge(left, right, on='key')   # the same as inner
print(left,'\n')
print(right,'\n')
print(result)
```

```
    A   B key
0  A0  B0  K0
1  A1  B1  K1
2  A2  B2  K2
3  A3  B3  K3
4  A4  B4  K4
5  A5  B5  K5

    C   D key
0  C0  D0  K0
1  C1  D1  K4
2  C2  D2  K3
3  C3  D3  K3

    A   B key   C   D
0  A0  B0  K0  C0  D0
1  A3  B3  K3  C2  D2
2  A3  B3  K3  C3  D3
3  A4  B4  K4  C1  D1
```

In [62]:
```
result = pd.merge(left, right, on='key',how='inner')
print(left,'\n')
print(right,'\n')
print(result)
```

```
    A   B key
0  A0  B0  K0
1  A1  B1  K1
2  A2  B2  K2
3  A3  B3  K3
4  A4  B4  K4
5  A5  B5  K5

    C   D key
0  C0  D0  K0
1  C1  D1  K4
2  C2  D2  K3
3  C3  D3  K3

    A   B key   C   D
0  A0  B0  K0  C0  D0
1  A3  B3  K3  C2  D2
2  A3  B3  K3  C3  D3
3  A4  B4  K4  C1  D1
```

In [63]:
```
result = pd.merge(left, right, on='key',how='right')
print(left,'\n')
print(right,'\n')
print(result)
```

```
    A   B key
0  A0  B0  K0
1  A1  B1  K1
2  A2  B2  K2
3  A3  B3  K3
4  A4  B4  K4
5  A5  B5  K5

    C   D key
0  C0  D0  K0
1  C1  D1  K4
2  C2  D2  K3
3  C3  D3  K3

    A   B key   C   D
0  A0  B0  K0  C0  D0
1  A3  B3  K3  C2  D2
2  A3  B3  K3  C3  D3
3  A4  B4  K4  C1  D1
```

In [64]:
```
result = pd.merge(left, right, on='key',how='left')
print(left,'\n')
print(right,'\n')
print(result)
```

```
    A   B key
0  A0  B0  K0
1  A1  B1  K1
2  A2  B2  K2
3  A3  B3  K3
4  A4  B4  K4
5  A5  B5  K5

    C   D key
0  C0  D0  K0
1  C1  D1  K4
2  C2  D2  K3
3  C3  D3  K3

    A   B key    C    D
0  A0  B0  K0   C0   D0
1  A1  B1  K1  NaN  NaN
2  A2  B2  K2  NaN  NaN
3  A3  B3  K3   C2   D2
4  A3  B3  K3   C3   D3
5  A4  B4  K4   C1   D1
6  A5  B5  K5  NaN  NaN
```

In [65]:
```
result = pd.merge(left, right, on='key',how='outer')
print(left,'\n')
print(right,'\n')
print(result)
```

```
    A   B key
0  A0  B0  K0
1  A1  B1  K1
2  A2  B2  K2
3  A3  B3  K3
4  A4  B4  K4
5  A5  B5  K5

    C   D key
0  C0  D0  K0
1  C1  D1  K4
2  C2  D2  K3
3  C3  D3  K3

    A   B key    C    D
0  A0  B0  K0   C0   D0
1  A1  B1  K1  NaN  NaN
2  A2  B2  K2  NaN  NaN
3  A3  B3  K3   C2   D2
4  A3  B3  K3   C3   D3
5  A4  B4  K4   C1   D1
6  A5  B5  K5  NaN  NaN
```

Merging based on more than one key

In [66]:
```
left = pd.DataFrame({'key1': ['K0', 'K1', 'K2', 'K3', 'K4', 'K5'],
                     'key2': ['k0', 'k1', 'k2', 'k0', 'k1', 'k2'],
                       'A': ['A0', 'A1', 'A2', 'A3', 'A4', 'A5'],
                       'B': ['B0', 'B1', 'B2', 'B3', 'B4', 'B5']})
right = pd.DataFrame({'key1': ['K0', 'K4', 'K3', 'K4'],
                      'key2': ['k0', 'k1', 'k2', 'k1'],
                        'C': ['C0', 'C1', 'C2', 'C3'],
                        'D': ['D0', 'D1', 'D2', 'D3']})
```

In [67]:
```
result = pd.merge(left, right, on=['key1', 'key2'])   # the same as inner
print(left,'\n')
print(right,'\n')
print(result)
```

```
    A   B key1 key2
0  A0  B0   K0   k0
1  A1  B1   K1   k1
2  A2  B2   K2   k2
3  A3  B3   K3   k0
4  A4  B4   K4   k1
5  A5  B5   K5   k2

    C   D key1 key2
0  C0  D0   K0   k0
1  C1  D1   K4   k1
2  C2  D2   K3   k2
3  C3  D3   K4   k1

    A   B key1 key2   C   D
0  A0  B0   K0   k0  C0  D0
1  A4  B4   K4   k1  C1  D1
2  A4  B4   K4   k1  C3  D3
```

In [68]:
```
result = pd.merge(left, right, how='inner', on=['key1', 'key2'])
print(left,'\n')
print(right,'\n')
print(result)
```

```
    A   B key1 key2
0  A0  B0   K0   k0
1  A1  B1   K1   k1
2  A2  B2   K2   k2
3  A3  B3   K3   k0
4  A4  B4   K4   k1
5  A5  B5   K5   k2

    C   D key1 key2
0  C0  D0   K0   k0
1  C1  D1   K4   k1
2  C2  D2   K3   k2
3  C3  D3   K4   k1

    A   B key1 key2   C   D
0  A0  B0   K0   k0  C0  D0
1  A4  B4   K4   k1  C1  D1
2  A4  B4   K4   k1  C3  D3
```

In [69]:
```
result = pd.merge(left, right, how='right', on=['key1', 'key2'])
print(left,'\n')
print(right,'\n')
print(result)
```

```
    A   B key1 key2
0  A0  B0   K0   k0
1  A1  B1   K1   k1
2  A2  B2   K2   k2
3  A3  B3   K3   k0
4  A4  B4   K4   k1
5  A5  B5   K5   k2

    C   D key1 key2
0  C0  D0   K0   k0
1  C1  D1   K4   k1
2  C2  D2   K3   k2
3  C3  D3   K4   k1

     A    B key1 key2   C   D
0   A0   B0   K0   k0  C0  D0
1   A4   B4   K4   k1  C1  D1
2   A4   B4   K4   k1  C3  D3
3  NaN  NaN   K3   k2  C2  D2
```

In [70]:
```
result = pd.merge(left, right, how='left', on=['key1', 'key2'])
print(left,'\n')
print(right,'\n')
print(result)
```

```
    A   B key1 key2
0  A0  B0   K0   k0
1  A1  B1   K1   k1
2  A2  B2   K2   k2
3  A3  B3   K3   k0
4  A4  B4   K4   k1
5  A5  B5   K5   k2

    C   D key1 key2
0  C0  D0   K0   k0
1  C1  D1   K4   k1
2  C2  D2   K3   k2
3  C3  D3   K4   k1

    A   B key1 key2    C    D
0  A0  B0   K0   k0   C0   D0
1  A1  B1   K1   k1  NaN  NaN
2  A2  B2   K2   k2  NaN  NaN
3  A3  B3   K3   k0  NaN  NaN
4  A4  B4   K4   k1   C1   D1
5  A4  B4   K4   k1   C3   D3
6  A5  B5   K5   k2  NaN  NaN
```

In [71]:
```
result = pd.merge(left, right, how='outer', on=['key1', 'key2'])
print(left,'\n')
print(right,'\n')
print(result)
```

```
    A   B key1 key2
0  A0  B0   K0   k0
1  A1  B1   K1   k1
2  A2  B2   K2   k2
3  A3  B3   K3   k0
4  A4  B4   K4   k1
5  A5  B5   K5   k2

    C   D key1 key2
0  C0  D0   K0   k0
1  C1  D1   K4   k1
2  C2  D2   K3   k2
3  C3  D3   K4   k1

     A    B key1 key2    C    D
0   A0   B0   K0   k0   C0   D0
1   A1   B1   K1   k1  NaN  NaN
2   A2   B2   K2   k2  NaN  NaN
3   A3   B3   K3   k0  NaN  NaN
4   A4   B4   K4   k1   C1   D1
5   A4   B4   K4   k1   C3   D3
6   A5   B5   K5   k2  NaN  NaN
7  NaN  NaN   K3   k2   C2   D2
```

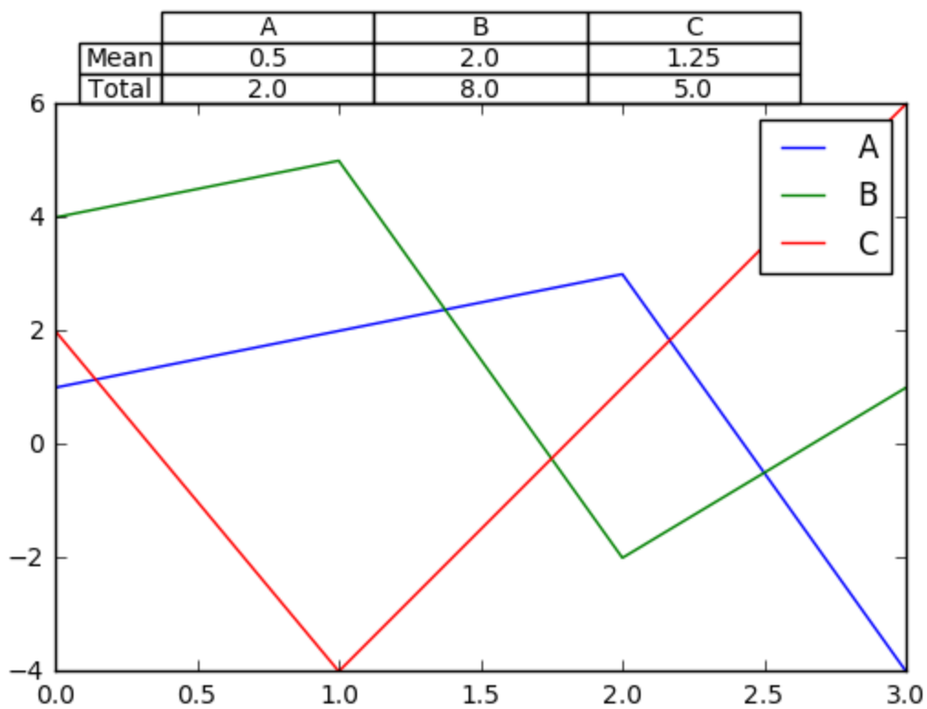
Formatting a Table

In [72]:
```
dc = pd.DataFrame({'A' : [1, 2, 3, -4],'B' : [4, 5, -2, 1],'C' : [2, -4, 1, 6]})

plt.plot(dc)
plt.legend(dc.columns)
dcsummary = pd.DataFrame([dc.mean(), dc.sum()],index=['Mean','Total'])

plt.table(cellText=dcsummary.values,colWidths = [0.25]*len(dc.columns),
          rowLabels=dcsummary.index,
          colLabels=dcsummary.columns,
          cellLoc = 'center', rowLoc = 'center',
          loc='top')
fig = plt.gcf()

plt.show()
```



In [73]:
```
print(dc.to_string())
```

```
   A  B  C
0  1  4  2
1  2  5 -4
2  3 -2  1
3 -4  1  6
```

In [74]:
```
HTML(dc.to_html())
```

Out[74]:

	A	B	C
0	1	4	2
1	2	5	-4
2	3	-2	1
3	-4	1	6

In [75]:
```
display(dc)
```

	A	B	C
0	1	4	2
1	2	5	-4
2	3	-2	1
3	-4	1	6

In [76]:
```
dc.style
```

Out[76]:

	A	B	C
0	1	4	2
1	2	5	-4
2	3	-2	1
3	-4	1	6

The following code from https://pandas.pydata.org/pandas-docs/stable/style.html (https://pandas.pydata.org/pandas-docs/stable/style.html)

In [77]:
```
def color_negative_red(val):
    """
    Takes a scalar and returns a string with
    the css property `'color: red'` for negative
    strings, black otherwise.
    """
    color = 'red' if val < 0 else 'black'
    return 'color: %s' % color


s = dc.style.applymap(color_negative_red)
s
```

Out[77]:

	A	B	C
0	1	4	2
1	2	5	-4
2	3	-2	1
3	-4	1	6

In [78]:
```
def highlight_max(data, color='yellow'):
    '''
    highlight the maximum in a Series or DataFrame
    '''
    attr = 'background-color: {}'.format(color)
    if data.ndim == 1:  # Series from .apply(axis=0) or axis=1
        is_max = data == data.max()
        return [attr if v else '' for v in is_max]
    else:  # from .apply(axis=None)
        is_max = data == data.max().max()
        return pd.DataFrame(np.where(is_max, attr, ''),
                            index=data.index, columns=data.columns)

dc.style.apply(highlight_max)
```

Out[78]:

	A	B	C
0	1	4	2
1	2	5	-4
2	3	-2	1
3	-4	1	6

We can use chain of commands. To highlight the max of each row, use axis=1

In [79]:
```
dc.style.\
    applymap(color_negative_red).\
    apply(highlight_max,color='lightgreen', axis=1)
```

Out[79]:

	A	B	C
0	1	4	2
1	2	5	-4
2	3	-2	1
3	-4	1	6

to highlight the max of all values, use axis=None

In [80]:
```
dc.style.apply(highlight_max, color='darkorange', axis=None)
```

Out[80]:

	A	B	C
0	1	4	2
1	2	5	-4
2	3	-2	1
3	-4	1	6